

Clustering con K-Means

Segmentación de Datos

Analítica de Datos, Universidad de San Andrés



¿Por qué Clustering?

Caso de uso: E-commerce con miles de clientes

- No todos los clientes son iguales
- Algunos compran seguido pero poco, otros compran mucho de una vez
- **Pregunta clave:** ¿Podemos agrupar automáticamente a clientes con comportamientos similares?
- **Beneficios:**
 - Personalizar campañas de marketing
 - Entender mejor qué tipo de clientes tenemos
 - Asignar recursos eficientemente

¿Qué es Clustering?

- Técnica de **aprendizaje no supervisado**
- Divide datos en grupos (clusters) donde:
 - Observaciones dentro del mismo cluster son **similares**
 - Observaciones de diferentes clusters son **diferentes**
- No necesitamos etiquetas previas
- Encontramos estructura oculta en los datos

- **Marketing:** Segmentación de clientes
- **Biología:** Clasificar genes o especies
- **Detección de fraude:** Identificar transacciones anómalas
- **Compresión:** Agrupar colores en imágenes
- **Recomendaciones:** Usuarios con gustos similares
- **Documentos:** Organizar noticias por tema

K-Means: La Idea Intuitiva

- Elegís cuántos grupos querés (el famoso “K”)

K-Means: La Idea Intuitiva

- Elegís cuántos grupos querés (el famoso “K”)
- El algoritmo coloca K puntos al azar (centroides iniciales)

K-Means: La Idea Intuitiva

- Elegís cuántos grupos querés (el famoso “K”)
- El algoritmo coloca K puntos al azar (centroides iniciales)
- Cada observación se asigna al centroide más cercano

K-Means: La Idea Intuitiva

- Elegís cuántos grupos querés (el famoso “K”)
- El algoritmo coloca K puntos al azar (centroides iniciales)
- Cada observación se asigna al centroide más cercano
- Se recalcula cada centroide como el promedio de sus puntos

K-Means: La Idea Intuitiva

- Elegís cuántos grupos querés (el famoso “K”)
- El algoritmo coloca K puntos al azar (centroides iniciales)
- Cada observación se asigna al centroide más cercano
- Se recalcula cada centroide como el promedio de sus puntos
- Se repite hasta que los centroides dejen de moverse

K-Means: La Idea Intuitiva

- Elegís cuántos grupos querés (el famoso “K”)
- El algoritmo coloca K puntos al azar (centroides iniciales)
- Cada observación se asigna al centroide más cercano
- Se recalcula cada centroide como el promedio de sus puntos
- Se repite hasta que los centroides dejen de moverse
- **Analogía:** K imanes que atraen puntos y se reposicionan

K-Means: Algoritmo Paso a Paso

Proceso iterativo:

1. **Inicialización:** Elegir K centroides iniciales al azar

K-Means: Algoritmo Paso a Paso

Proceso iterativo:

1. **Inicialización:** Elegir K centroides iniciales al azar
2. **Asignación:** Para cada observación x_i , asignarla al cluster cuyo centroide esté más cerca

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

K-Means: Algoritmo Paso a Paso

Proceso iterativo:

1. **Inicialización:** Elegir K centroides iniciales al azar
2. **Asignación:** Para cada observación x_i , asignarla al cluster cuyo centroide esté más cerca

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

3. **Actualización:** Recalcular cada centroide como el promedio de sus observaciones

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$$

K-Means: Algoritmo Paso a Paso

Proceso iterativo:

1. **Inicialización:** Elegir K centroides iniciales al azar
2. **Asignación:** Para cada observación x_i , asignarla al cluster cuyo centroide esté más cerca

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

3. **Actualización:** Recalcular cada centroide como el promedio de sus observaciones

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$$

4. **Repetir:** Pasos 2 y 3 hasta convergencia

Ejemplo Paso a Paso: Datos

6 clientes con 2 features cada uno

Cliente	Gasto Promedio	Frecuencia (veces/mes)
A	\$20	2
B	\$25	3
C	\$100	8
D	\$95	9
E	\$22	2
F	\$105	10

Objetivo: Dividir en $K=2$ clusters

Ejemplo Paso a Paso: Inicialización

Paso 1: Elegir 2 centroides iniciales al azar

- Centroide 1 (μ_1): Cliente A = (\$20, 2)
- Centroide 2 (μ_2): Cliente C = (\$100, 8)

Ejemplo Paso a Paso: Inicialización

Paso 1: Elegir 2 centroides iniciales al azar

- Centroide 1 (μ_1): Cliente A = (\$20, 2)
- Centroide 2 (μ_2): Cliente C = (\$100, 8)

Ejemplo Paso a Paso: Inicialización

Paso 1: Elegir 2 centroides iniciales al azar

- Centroide 1 (μ_1): Cliente A = (\$20, 2)
- Centroide 2 (μ_2): Cliente C = (\$100, 8)

Nota: En la práctica, sklearn elige los centroides de forma más inteligente

Ejemplo Paso a Paso: Asignación

Paso 2: Asignar cada cliente al centroide más cercano

Cliente B (\$25, 3):

- Distancia a μ_1 : $\sqrt{(25 - 20)^2 + (3 - 2)^2} = \sqrt{26} \approx 5,1$

Ejemplo Paso a Paso: Asignación

Paso 2: Asignar cada cliente al centroide más cercano

Cliente B (\$25, 3):

- Distancia a μ_1 : $\sqrt{(25 - 20)^2 + (3 - 2)^2} = \sqrt{26} \approx 5,1$
- Distancia a μ_2 : $\sqrt{(25 - 100)^2 + (3 - 8)^2} = \sqrt{5650} \approx 75,2$

Ejemplo Paso a Paso: Asignación

Paso 2: Asignar cada cliente al centroide más cercano

Cliente B (\$25, 3):

- Distancia a μ_1 : $\sqrt{(25 - 20)^2 + (3 - 2)^2} = \sqrt{26} \approx 5,1$
- Distancia a μ_2 : $\sqrt{(25 - 100)^2 + (3 - 8)^2} = \sqrt{5650} \approx 75,2$
- B está más cerca de $\mu_1 \rightarrow$ **Cluster 1**

Ejemplo Paso a Paso: Asignación

Paso 2: Asignar cada cliente al centroide más cercano

Cliente B (\$25, 3):

- Distancia a μ_1 : $\sqrt{(25 - 20)^2 + (3 - 2)^2} = \sqrt{26} \approx 5,1$
- Distancia a μ_2 : $\sqrt{(25 - 100)^2 + (3 - 8)^2} = \sqrt{5650} \approx 75,2$
- B está más cerca de $\mu_1 \rightarrow$ **Cluster 1**

Haciendo esto para todos:

- Cluster 1: A, B, E (bajo gasto, baja frecuencia)
- Cluster 2: C, D, F (alto gasto, alta frecuencia)

Ejemplo Paso a Paso: Actualización

Paso 3: Recalcular centroides como promedio de cada cluster

Cluster 1 (A, B, E):

$$\begin{aligned}\mu_1 &= \left(\frac{20 + 25 + 22}{3}, \frac{2 + 3 + 2}{3} \right) \\ &= (22, 3, 2, 3)\end{aligned}$$

Cluster 2 (C, D, F):

$$\begin{aligned}\mu_2 &= \left(\frac{100 + 95 + 105}{3}, \frac{8 + 9 + 10}{3} \right) \\ &= (100, 9)\end{aligned}$$

Ejemplo Paso a Paso: Convergencia

Paso 4: Repetir hasta que los centroides no cambien

- Volvemos al Paso 2 con los nuevos centroides

Ejemplo Paso a Paso: Convergencia

Paso 4: Repetir hasta que los centroides no cambien

- Volvemos al Paso 2 con los nuevos centroides
- Calculamos distancias y reasignamos clientes

Ejemplo Paso a Paso: Convergencia

Paso 4: Repetir hasta que los centroides no cambien

- Volvemos al Paso 2 con los nuevos centroides
- Calculamos distancias y reasignamos clientes
- En este caso, las asignaciones no cambian

Ejemplo Paso a Paso: Convergencia

Paso 4: Repetir hasta que los centroides no cambien

- Volvemos al Paso 2 con los nuevos centroides
- Calculamos distancias y reasignamos clientes
- En este caso, las asignaciones no cambian
- **¡Convergencia alcanzada!**

Ejemplo Paso a Paso: Convergencia

Paso 4: Repetir hasta que los centroides no cambien

- Volvemos al Paso 2 con los nuevos centroides
- Calculamos distancias y reasignamos clientes
- En este caso, las asignaciones no cambian
- **¡Convergencia alcanzada!**

Resultado final:

- Cluster 1: Clientes ocasionales (bajo gasto y frecuencia)
- Cluster 2: Clientes VIP (alto gasto y frecuencia)

¿Cómo Elegir K?

Problema: No sabemos de antemano cuántos clusters hay

- A diferencia del aprendizaje supervisado, no tenemos conjunto de test

¿Cómo Elegir K?

Problema: No sabemos de antemano cuántos clusters hay

- A diferencia del aprendizaje supervisado, no tenemos conjunto de test
- Necesitamos métodos para determinar el K óptimo

Método del Codo (Elbow Method)

- Probamos diferentes valores de K

Método del Codo (Elbow Method)

- Probamos diferentes valores de K
- Para cada K, calculamos la **inercia**:

$$\text{Inercia} = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

Método del Codo (Elbow Method)

- Probamos diferentes valores de K
- Para cada K, calculamos la **inercia**:

$$\text{Inercia} = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

- Graficamos inercia vs K

Método del Codo (Elbow Method)

- Probamos diferentes valores de K
- Para cada K, calculamos la **inercia**:

$$\text{Inercia} = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

- Graficamos inercia vs K
- Buscamos el “codo”: donde agregar más clusters no reduce mucho la inercia

Método del Codo (Elbow Method)

- Probamos diferentes valores de K
- Para cada K, calculamos la **inercia**:

$$\text{Inercia} = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

- Graficamos inercia vs K
- Buscamos el “codo”: donde agregar más clusters no reduce mucho la inercia
- **Intuición:** Punto de rendimientos decrecientes

Mide qué tan bien está cada observación asignada

- Para cada observación i :
 - a_i = distancia promedio a puntos en su cluster
 - b_i = distancia promedio al cluster más cercano

Silhouette Score

Mide qué tan bien está cada observación asignada

- Para cada observación i :
 - a_i = distancia promedio a puntos en su cluster
 - b_i = distancia promedio al cluster más cercano
- Silhouette Score:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Silhouette Score

Mide qué tan bien está cada observación asignada

- Para cada observación i :
 - a_i = distancia promedio a puntos en su cluster
 - b_i = distancia promedio al cluster más cercano

- Silhouette Score:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- Rango: -1 a 1
 - Cercano a 1: bien asignado
 - Cercano a 0: en el borde
 - Negativo: probablemente mal asignado

Silhouette Score

Mide qué tan bien está cada observación asignada

- Para cada observación i :
 - a_i = distancia promedio a puntos en su cluster
 - b_i = distancia promedio al cluster más cercano

- Silhouette Score:

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- Rango: -1 a 1
 - Cercano a 1: bien asignado
 - Cercano a 0: en el borde
 - Negativo: probablemente mal asignado
- Elegimos el K que maximice el Silhouette Score promedio

Implementación: K-Means Básico

```
1 from sklearn.cluster import KMeans
2
3 # Crear el modelo con K=3
4 kmeans = KMeans(n_clusters=3, random_state=42)
5
6 # Entrenar y obtener las asignaciones
7 clusters = kmeans.fit_predict(X)
8
9 # Obtener los centroides
10 centroides = kmeans.cluster_centers_
11
12 # Agregar clusters al DataFrame
13 df['cluster'] = clusters
```

Implementación: Método del Codo

```
1 # Probar diferentes valores de K
2 inercias = []
3 K_range = range(1, 11)
4
5 for k in K_range:
6     kmeans = KMeans(n_clusters=k, random_state=42)
7     kmeans.fit(X)
8     inercias.append(kmeans.inertia_)
9
10 # Graficar
11 plt.plot(K_range, inercias, marker='o')
12 plt.xlabel('Numero de Clusters (K)')
13 plt.ylabel('Inercia')
14 plt.title('Metodo del Codo')
15 plt.show()
```

Implementación: Silhouette Score

```
1 from sklearn.metrics import silhouette_score
2
3 # Calcular Silhouette para diferentes K
4 silhouette_scores = []
5 K_range = range(2, 11)
6
7 for k in K_range:
8     kmeans = KMeans(n_clusters=k, random_state=42)
9     clusters = kmeans.fit_predict(X)
10    score = silhouette_score(X, clusters)
11    silhouette_scores.append(score)
12
13 # Graficar
14 plt.plot(K_range, silhouette_scores, marker='o')
15 plt.xlabel('Numero de Clusters (K)')
16 plt.ylabel('Silhouette Score')
17 plt.show()
```

Analizar los centroides:

- Los centroides representan el “cliente típico” de cada cluster

Analizar los centroides:

- Los centroides representan el “cliente típico” de cada cluster
- Ejemplo de interpretación:
 - Cluster 0: \$25, 2 veces/mes → Ocasionales
 - Cluster 1: \$150, 10 veces/mes → VIP
 - Cluster 2: \$60, 5 veces/mes → Regulares

Analizar los centroides:

- Los centroides representan el “cliente típico” de cada cluster
- Ejemplo de interpretación:
 - Cluster 0: \$25, 2 veces/mes → Ocasionales
 - Cluster 1: \$150, 10 veces/mes → VIP
 - Cluster 2: \$60, 5 veces/mes → Regulares
- **Acción:** Diseñar campañas específicas por segmento

Ventajas de K-Means

- **Simple y rápido:** Fácil de entender e implementar

Ventajas de K-Means

- **Simple y rápido:** Fácil de entender e implementar
- **Escalable:** Funciona bien con datasets grandes

Ventajas de K-Means

- **Simple y rápido:** Fácil de entender e implementar
- **Escalable:** Funciona bien con datasets grandes
- **Garantía de convergencia:** Siempre termina

Ventajas de K-Means

- **Simple y rápido:** Fácil de entender e implementar
- **Escalable:** Funciona bien con datasets grandes
- **Garantía de convergencia:** Siempre termina
- **Interpretable:** Centroides fáciles de analizar

Limitaciones de K-Means

- **Elegir K:** No es obvio cuál es el mejor K

Limitaciones de K-Means

- **Elegir K:** No es obvio cuál es el mejor K
- **Inicialización:** Resultados pueden variar según inicialización

Limitaciones de K-Means

- **Elegir K:** No es obvio cuál es el mejor K
- **Inicialización:** Resultados pueden variar según inicialización
- **Clusters esféricos:** Asume clusters con forma esférica

Limitaciones de K-Means

- **Elegir K:** No es obvio cuál es el mejor K
- **Inicialización:** Resultados pueden variar según inicialización
- **Clusters esféricos:** Asume clusters con forma esférica
- **Outliers:** Sensible a valores extremos

Limitaciones de K-Means

- **Elegir K:** No es obvio cuál es el mejor K
- **Inicialización:** Resultados pueden variar según inicialización
- **Clusters esféricos:** Asume clusters con forma esférica
- **Outliers:** Sensible a valores extremos
- **Tamaño similar:** Asume clusters de tamaño parecido

Normalización de Features

- K-Means usa distancia euclidiana

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X_scaled = scaler.fit_transform(X)
5
6 kmeans = KMeans(n_clusters=3, random_state=42)
7 clusters = kmeans.fit_predict(X_scaled)
```

Normalización de Features

- K-Means usa distancia euclidiana
- Features con rangos grandes dominan el cálculo

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X_scaled = scaler.fit_transform(X)
5
6 kmeans = KMeans(n_clusters=3, random_state=42)
7 clusters = kmeans.fit_predict(X_scaled)
```

Normalización de Features

- K-Means usa distancia euclidiana
- Features con rangos grandes dominan el cálculo
- **Solución:** Estandarizar antes de aplicar K-Means

```
1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X_scaled = scaler.fit_transform(X)
5
6 kmeans = KMeans(n_clusters=3, random_state=42)
7 clusters = kmeans.fit_predict(X_scaled)
```

¿Cuándo NO Usar K-Means?

- Clusters con formas no esféricas (ej: media luna)

¿Cuándo NO Usar K-Means?

- Clusters con formas no esféricas (ej: media luna)
- Muchos outliers que distorsionarían los centroides

¿Cuándo NO Usar K-Means?

- Clusters con formas no esféricas (ej: media luna)
- Muchos outliers que distorsionarían los centroides
- Necesitas detectar el número de clusters automáticamente

¿Cuándo NO Usar K-Means?

- Clusters con formas no esféricas (ej: media luna)
- Muchos outliers que distorsionarían los centroides
- Necesitas detectar el número de clusters automáticamente
- **Alternativas:** DBSCAN, Hierarchical Clustering, HDBSCAN

- **Clustering:** Agrupar datos sin etiquetas previas

- **Clustering:** Agrupar datos sin etiquetas previas
- **K-Means:** Algoritmo iterativo simple y eficiente

- **Clustering:** Agrupar datos sin etiquetas previas
- **K-Means:** Algoritmo iterativo simple y eficiente
- **Elegir K:** Método del codo + Silhouette Score

- **Clustering:** Agrupar datos sin etiquetas previas
- **K-Means:** Algoritmo iterativo simple y eficiente
- **Elegir K:** Método del codo + Silhouette Score
- **Normalización:** SIEMPRE estandarizar features

- **Clustering:** Agrupar datos sin etiquetas previas
- **K-Means:** Algoritmo iterativo simple y eficiente
- **Elegir K:** Método del codo + Silhouette Score
- **Normalización:** SIEMPRE estandarizar features
- **Interpretación:** Analizar centroides para entender segmentos

- **Clustering:** Agrupar datos sin etiquetas previas
- **K-Means:** Algoritmo iterativo simple y eficiente
- **Elegir K:** Método del codo + Silhouette Score
- **Normalización:** SIEMPRE estandarizar features
- **Interpretación:** Analizar centroides para entender segmentos
- **Aplicación:** Segmentación de clientes, detección de patrones, compresión

¿Dudas?
¿Consultas?



Universidad de
SanAndrés